



Guia de instalação e configuração

Ubuntu Server 24.04 LTS em VPS na AWS (EC2) – passo a passo

Abrange: criação da instância EC2, preparação do Ubuntu, instalação com Docker ou manual, Nginx + HTTPS (Let's Encrypt), e-mail (Amazon SES), primeiro acesso e migração do Clockify, backups no S3, atualização, RDS, segurança e solução de problemas

Repositório: [melhorartecnologia-arch/clockfy](https://github.com/melhorartecnologia-arch/clockfy)

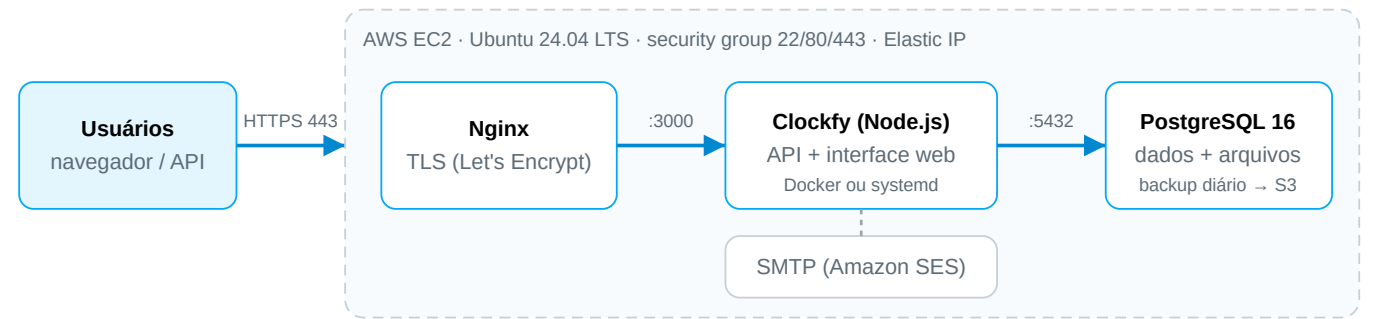
Data: 29/09/2026

Sumário

1	Visão geral e requisitos	8	E-mail (SMTP) – convites, lembretes, relatórios e faturas
2	Criar a instância EC2 (console AWS)	9	Primeiro acesso e configuração inicial
3	Conectar ao servidor por SSH	10	Backups e restauração
4	Preparar o Ubuntu 24.04	11	Atualizar o Clockfy
5	Caminho A – Instalação com Docker (recomendado)	12	Operação: logs, monitoramento e comandos úteis
6	Caminho B – Instalação manual (Node.js + PostgreSQL + systemd)	13	Opcional: banco gerenciado no Amazon RDS
7	Nginx como proxy reverso e HTTPS (Let's Encrypt)	14	Segurança: lista de verificação
		15	Solução de problemas
		16	Apêndice: arquivos de implantação

1. Visão geral e requisitos

Este guia mostra, passo a passo, como colocar o **Clockfy** em produção em uma VPS **Ubuntu Server 24.04 LTS** na **AWS (EC2)**, com HTTPS, e-mail, backups automáticos e atualização. Há dois caminhos de instalação: **A) Docker Compose** (recomendado, mais simples de manter) e **B) manual** (Node.js + PostgreSQL instalados no sistema e serviço systemd). Escolha um dos dois; as demais seções (Nginx/HTTPS, e-mail, backups, atualização) valem para ambos.



Dimensionamento sugerido

Cenário	Instância EC2	Disco	Observações
Até ~25 usuários	t3.small (2 vCPU, 2 GB)	30 GB gp3	Crie 2 GB de swap para a compilação da interface (seção 4).
Até ~150 usuários	t3.medium (2 vCPU, 4 GB)	40 GB gp3	Confortável para relatórios grandes e importação do Clockfy.
Acima disso / alta disponibilidade	t3.large + Amazon RDS PostgreSQL	40 GB + RDS	Banco gerenciado com backups automáticos (seção 13).

O que você precisa antes de começar

- Uma conta AWS com permissão para criar instâncias EC2 (e, opcionalmente, Route 53 e SES).

- Um domínio ou subdomínio para o sistema, ex.: `clockfy.suaempresa.com.br` (necessário para HTTPS e para os links de convite/relatórios).
- Acesso ao repositório do projeto: `https://github.com/melhorartecnologia-arch/clockfy` (branch `claude/zen-curie-y7y4bq` ou `main` após a integração).
- Um cliente SSH (Terminal no macOS/Linux; PowerShell ou PuTTY no Windows).

Convenções deste guia

Comandos começam com `$` e são executados no servidor como usuário `ubuntu` (com `sudo` quando indicado). Substitua `clockfy.suaempresa.com.br` pelo seu domínio e `SEU_IP` pelo IP elástico da instância. Os arquivos citados (`deploy/nginx-clockfy.conf`, `deploy/clockfy.service`, `deploy/backup.sh` ...) fazem parte do repositório.

2. Criar a instância EC2 (console AWS)

1. Acesse **console.aws.amazon.com** > **EC2** e escolha a região mais próxima dos usuários (ex.: **South America (São Paulo) sa-east-1**).
2. Clique em **Launch instance**. Em **Name**, informe `clockfy`.
3. Em **Application and OS Images**, selecione **Ubuntu Server 24.04 LTS (HVM), SSD Volume Type**, arquitetura **64-bit (x86)**.
4. Em **Instance type**, escolha `t3.small` (ou `t3.medium`, conforme a tabela da seção 1).
5. Em **Key pair (login)**, clique em **Create new key pair**: nome `clockfy-key`, tipo RSA, formato `.pem`. Baixe e guarde o arquivo – ele não pode ser baixado de novo.
6. Em **Network settings** > **Edit**: mantenha a VPC padrão, **Auto-assign public IP: Enable**. Crie um security group chamado `clockfy-sg` com as regras de entrada: **SSH (22)** apenas do seu IP (*My IP*), **HTTP (80)** de *Anywhere* (0.0.0.0/0) e **HTTPS (443)** de *Anywhere*. Não abra a porta 5432 nem a 3000.
7. Em **Configure storage**, defina **30 GiB** do tipo **gp3**.
8. Clique em **Launch instance** e aguarde o estado *Running* com *2/2 checks passed*.
9. **IP fixo**: em **EC2** > **Network & Security** > **Elastic IPs**, clique em **Allocate Elastic IP address** e depois em **Actions** > **Associate**, escolhendo a instância `clockfy`. Anote o IP (`SEU_IP`).
10. **DNS**: no seu provedor de DNS (ou **Route 53** > **Hosted zones**), crie um registro **A** para `clockfy.suaempresa.com.br` apontando para `SEU_IP` (TTL 300). Aguarde a propagação (normalmente minutos).

Boa prática

Ative o **Termination protection** (Actions > Instance settings > Change termination protection) e crie uma tag `Backup=true` para políticas de snapshot do EBS (AWS Backup ou Data Lifecycle Manager).

3. Conectar ao servidor por SSH

No macOS/Linux (Terminal) ou Windows (PowerShell):

```
chmod 400 ~/Downloads/clockfy-key.pem          # só no macOS/Linux
ssh -i ~/Downloads/clockfy-key.pem ubuntu@SEU_IP
```

No Windows com PuTTY, converta o `.pem` para `.ppk` com o PuTTYgen e use o usuário `ubuntu`. Na primeira conexão, confirme a impressão digital do host com **yes**.

Se a conexão falhar

Confira se a regra SSH do security group contém o seu IP atual (ele muda em redes domésticas/móveis), se o IP usado é o Elastic IP e se a permissão do arquivo `.pem` é 400.

4. Preparar o Ubuntu 24.04

4.1 Atualizações, fuso horário e hostname

```
sudo apt update && sudo apt -y upgrade
sudo timedatectl set-timezone America/Sao_Paulo
sudo hostnamectl set-hostname clockfy
sudo apt -y install git curl ufw fail2ban unattended-upgrades ca-certificates gnupg
sudo dpkg-reconfigure -plow unattended-upgrades      # escolha "Yes" para atualizações de segurança
automáticas
sudo reboot      # se o upgrade pedir reinicialização (reconecte após ~1 minuto)
```

4.2 Swap (recomendado em instâncias de 2 GB)

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h      # deve mostrar 2.0Gi em Swap
```

4.3 Firewall (ufw) e proteção contra força bruta

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable && sudo ufw status verbose
sudo systemctl enable --now fail2ban
```

O ufw complementa o security group da AWS: as portas 3000 (aplicação) e 5432 (banco) ficam acessíveis apenas dentro do servidor.

5. Caminho A – Instalação com Docker (recomendado)

5.1 Instalar o Docker Engine e o Compose

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu noble stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null
sudo apt update && sudo apt -y install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo usermod -aG docker ubuntu && newgrp docker
docker --version && docker compose version
```

5.2 Obter o código

```
sudo mkdir -p /opt/clockfy && sudo chown ubuntu:ubuntu /opt/clockfy
git clone -b claude/zen-curie-y7y4bq https://github.com/melhorartecnologia-arch/clockfy.git
/opt/clockfy
cd /opt/clockfy
```

Se o repositório for privado, gere um token no GitHub (Settings > Developer settings > Personal access tokens) e use `https://TOKEN@github.com/melhorartecnologia-arch/clockfy.git`, ou configure uma chave SSH de deploy.

5.3 Configurar o arquivo .env

```
cp .env.example .env
openssl rand -hex 32      # copie o resultado para JWT_SECRET
nano .env
```

Edite pelo menos estas linhas (as demais podem ficar com o padrão):

```
JWT_SECRET=cole-aqui-o-valor-gerado-pelo-openssl
APP_URL=https://clockfy.suaempresa.com.br
# e-mail (opcional agora; veja a seção 8)
SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>
```

No modo Docker, o banco de dados é criado automaticamente pelo `docker-compose.yml` (usuário `clockfy`, senha `clockfy`, acessível só de dentro do servidor). Para trocar a senha do banco, altere `POSTGRES_PASSWORD` e `DATABASE_URL` no `docker-compose.yml` antes da primeira subida.

5.4 Subir a aplicação

```
docker compose up -d --build      # compila a interface e sobe app + PostgreSQL (3-6 min na primeira vez)
docker compose ps                 # app e db devem estar "running (healthy)"
curl -s http://localhost:3000/health # {"status":"ok",...}
docker compose logs -f app        # Ctrl+C para sair; procure "[clockfy] listening on http://localhost:3000"
```

As migrações do banco são aplicadas automaticamente na inicialização. Os dados ficam nos volumes Docker `pgdata` e `uploads`, que sobrevivem a atualizações e reinicializações. Os contêineres reiniciam sozinhos após um reboot do servidor (`restart: unless-stopped`).

Siga agora para a seção 7 (Nginx e HTTPS).

6. Caminho B – Instalação manual (Node.js + PostgreSQL + systemd)

6.1 Node.js 22

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt -y install nodejs
node -v && npm -v          # v22.x
```

6.2 PostgreSQL 16

```
sudo apt -y install postgresql postgresql-contrib
sudo systemctl enable --now postgresql
sudo -u postgres psql -c "CREATE USER clockfy WITH PASSWORD 'TROQUE_ESTA_SENHA';"
sudo -u postgres psql -c "CREATE DATABASE clockfy OWNER clockfy ENCODING 'UTF8';"
```

6.3 Usuário do serviço e código

```
sudo adduser --system --group --home /opt/clockfy --shell /bin/bash clockfy
sudo git clone -b claude/zen-curie-y7y4bq https://github.com/melhorartecnologia-arch/clockfy.git /opt/clockfy/src
sudo mv /opt/clockfy/src/* /opt/clockfy/src/.[*] /opt/clockfy/ 2>/dev/null; sudo rmdir /opt/clockfy/src
sudo chown -R clockfy:clockfy /opt/clockfy
```

6.4 Configuração (.env em server/)

```
sudo -u clockfy cp /opt/clockfy/.env.example /opt/clockfy/server/.env
openssl rand -hex 32
sudo -u clockfy nano /opt/clockfy/server/.env
```

```

PORT=3000
NODE_ENV=production
DATABASE_URL=postgres://clockfy:TROQUE_ESTA_SENHA@localhost:5432/clockfy
JWT_SECRET=valor-gerado-pelo-openssl
APP_URL=https://clockfy.suaempresa.com.br
SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>

```

6.5 Instalar dependências, compilar e migrar

```

cd /opt/clockfy
sudo -u clockfy npm install           # instala server/ e web/
sudo -u clockfy npm run build         # gera web/dist (interface)
sudo -u clockfy npm run migrate       # cria as tabelas

```

6.6 Serviço systemd

```

sudo cp /opt/clockfy/deploy/clockfy.service /etc/systemd/system/clockfy.service
sudo systemctl daemon-reload
sudo systemctl enable --now clockfy
sudo systemctl status clockfy --no-pager
curl -s http://localhost:3000/health
sudo journalctl -u clockfy -f          # logs em tempo real

```

O serviço reinicia automaticamente em caso de falha e após reboot. Siga para a seção 7.

7. Nginx como proxy reverso e HTTPS (Let's Encrypt)

7.1 Instalar e configurar o Nginx

```

sudo apt -y install nginx
sudo cp /opt/clockfy/deploy/nginx-clockfy.conf /etc/nginx/sites-available/clockfy
sudo sed -i 's/clockfy.exemplo.com.br/clockfy.suaempresa.com.br/g' /etc/nginx/sites-available/clockfy
sudo ln -s /etc/nginx/sites-available/clockfy /etc/nginx/sites-enabled/clockfy
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx

```

Teste em `http://clockfy.suaempresa.com.br` – a tela de login deve aparecer (ainda sem cadeado).

7.2 Certificado HTTPS gratuito com o Certbot

```

sudo snap install core && sudo snap refresh core
sudo snap install --classic certbot
sudo ln -sf /snap/bin/certbot /usr/bin/certbot
sudo certbot --nginx -d clockfy.suaempresa.com.br --redirect -m seu-email@suaempresa.com.br --agree-tos -n
sudo certbot renew --dry-run          # testa a renovação automática (o snap agenda a renovação sozinho)

```

O Certbot altera o arquivo do Nginx para escutar na 443 com o certificado e redirecionar HTTP para HTTPS. A renovação ocorre automaticamente a cada ~60 dias.

Confira o APP_URL

Depois do HTTPS, garanta que `APP_URL=https://clockfy.suaempresa.com.br` no `.env` e reinicie a aplicação (`docker compose up -d` OU `sudo systemctl restart clockfy`). Os links de convite, de relatórios compartilhados e do quiosque usam esse valor.

8. E-mail (SMTP) – convites, lembretes, relatórios e faturas

Sem SMTP o sistema funciona normalmente (os links de convite podem ser copiados na tela de equipe e as notificações ficam no app), mas e-mails de convite, lembretes, alertas, relatórios agendados e faturas só são enviados com um servidor SMTP configurado.

Opção 1 – Amazon SES (recomendado na AWS)

1. No console **Amazon SES** (mesma região ou *us-east-1*), em **Identities > Create identity**, verifique o seu domínio (adicione os registros DKIM no DNS) ou um endereço de e-mail.
2. Em **SMTP settings > Create SMTP credentials**, gere o usuário e a senha SMTP (guarde-os; são diferentes das chaves IAM).
3. Enquanto a conta estiver em *sandbox*, só é possível enviar para endereços verificados; solicite a saída do sandbox em **Account dashboard > Request production access**.
4. Configure no `.env` :

```
SMTP_HOST=email-smtp.sa-east-1.amazonaws.com    # use o endpoint da região do SES
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=AKIAxxxxxxxxxxxxxxxxxx
SMTP_PASS=BFxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>
```

Opção 2 – Google Workspace / Gmail

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=no-reply@suaempresa.com.br
SMTP_PASS=senha-de-app-de-16-caracteres        # Conta Google > Segurança > Senhas de app (requer 2FA)
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>
```

Após alterar o `.env`, reinicie a aplicação e teste convidando um membro em **Equipe > Convidar membros**. Falhas de envio aparecem nos logs com o prefixo `[mail]`.

9. Primeiro acesso e configuração inicial

1. Abra `https://clockfy.suaempresa.com.br` e clique em **Criar conta**. O primeiro usuário vira proprietário/administrador do workspace criado.

2. Em **Configurações > Geral**: nome do workspace, moeda (ex.: BRL), taxa horária e custo padrão, dias úteis e capacidade diária, formatos.
3. Em **Configurações > Permissões** e **Campos obrigatórios**: defina quem pode criar projetos/etiquetas/tarefas, quem vê taxas e o que é obrigatório ao registrar tempo.
4. Em **Configurações > Funcionalidades**: ligue/desligue aprovações, folgas, agenda, despesas, faturas e quiosque.
5. Cadastre **Clientes**, **Projetos** (com tarefas e estimativas) e **Etiquetas** – ou importe tudo do Clockify (próximo passo).
6. **Migração**: em **Administração > Importar do Clockify**, informe a chave de API do Clockify (Profile settings > API), escolha o workspace de origem e execute. Reexecute com “desde” para trazer registros novos até desligar o Clockify.
7. Convide a equipe em **Equipe > Convidar membros** e defina papéis (admin, gerente de equipe, gerente de projeto).
8. Gere chaves de API em **Perfil > Chaves de API** para integrações; aponte-as para `https://clockfy.suaempresa.com.br/api/v1`.

Dados de demonstração (opcional, só em ambientes de teste)

Docker: `docker compose exec app node server/src/cli/seed.js` · Manual: `cd /opt/clockfy && sudo -u clockfy npm run seed`. Cria o usuário `admin@clockfy.local` / `admin123` com projetos e registros de exemplo. Não use em produção.

10. Backups e restauração

Todos os dados (inclusive recibos e logotipos) ficam no PostgreSQL, portanto um dump do banco é um backup completo. O script `deploy/backup.sh` funciona nos dois modos de instalação, mantém 14 dias de arquivos e pode enviar para o S3.

10.1 Backup diário automático

```
sudo mkdir -p /var/backups/clockfy
sudo /opt/clockfy/deploy/backup.sh # teste manual
sudo crontab -e # adicione a linha abaixo (todo dia às 03:00)
0 3 * * * /opt/clockfy/deploy/backup.sh >> /var/log/clockfy-backup.log 2>&1
```

10.2 Enviar para o Amazon S3 (recomendado)

1. Crie um bucket (ex.: `suaempresa-backups`) com *Block public access* ativado e uma regra de ciclo de vida para expirar objetos após 90 dias.
2. Crie uma **IAM role** para EC2 com permissão `s3:PutObject` no bucket e associe-a à instância (Actions > Security > Modify IAM role) – sem chaves no servidor.
3. Instale a CLI e defina o bucket no cron:

```
sudo snap install aws-cli --classic
0 3 * * * S3_BUCKET=s3://suaempresa-backups/clockfy /opt/clockfy/deploy/backup.sh >> /var/log/clockfy-backup.log 2>&1
```

10.3 Restaurar

```
ls -lh /var/backups/clockfy/
sudo /opt/clockfy/deploy/restore.sh /var/backups/clockfy/clockfy-20260929-030000.sql.gz
```

Complementarmente, ative snapshots automáticos do volume EBS (AWS Backup ou Data Lifecycle Manager) para recuperação do servidor inteiro.

11. Atualizar o Clockfy

```
sudo /opt/clockfy/deploy/backup.sh          # sempre faça backup antes
cd /opt/clockfy && ./deploy/update.sh        # Docker: git pull + rebuild; manual: use MODE=manual sudo
-E ./deploy/update.sh
```

O script faz `git pull`, recompila a interface e reinicia o serviço; as migrações novas são aplicadas automaticamente na inicialização. Para voltar a uma versão anterior, faça `git checkout <commit>`, execute o mesmo script e, se necessário, restaure o backup.

12. Operação: logs, monitoramento e comandos úteis

Tarefa	Docker (caminho A)	Manual (caminho B)
Ver logs	<code>docker compose logs -f app</code>	<code>sudo journalctl -u clockfy -f</code>
Reiniciar	<code>docker compose restart app</code>	<code>sudo systemctl restart clockfy</code>
Parar / iniciar	<code>docker compose stop / docker compose start</code>	<code>sudo systemctl stop start clockfy</code>
Status	<code>docker compose ps</code>	<code>systemctl status clockfy</code>
Acessar o banco	<code>docker compose exec db psql -U clockfy -d clockfy</code>	<code>sudo -u postgres psql -d clockfy</code>
Alterar configuração	editar <code>/opt/clockfy/.env</code> e <code>docker compose up -d</code>	editar <code>/opt/clockfy/server/.env</code> e reiniciar o serviço
Verificação de saúde	<code>curl -s https://clockfy.suaempresa.com.br/health</code>	idem
Espaço em disco	<code>df -h · docker system df · docker image prune -f</code>	<code>df -h</code>

Para alertas de indisponibilidade, use um monitor externo (Amazon CloudWatch Synthetics, UptimeRobot ou similar) apontando para `/health`. Para métricas do servidor, o **CloudWatch agent** pode ser instalado opcionalmente.

13. Opcional: banco gerenciado no Amazon RDS

1. Em **RDS > Create database**: PostgreSQL 16, template *Production* ou *Dev/Test*, instância `db.t4g.micro` ou superior, armazenamento gp3 20 GB, **Public access**: **No**, mesma VPC da EC2.
2. No security group do RDS, libere a porta 5432 apenas para o security group `clockfy-sg` da EC2.
3. Crie o banco `clockfy` (nome inicial do banco na criação) e anote endpoint, usuário e senha.
4. Configure `DATABASE_URL=postgres://usuario:senha@ENDPOINT.rds.amazonaws.com:5432/clockfy?sslmode=no-verify` (o RDS exige TLS por padrão) e reinicie a aplicação.
5. No Docker, remova o serviço `db` e o `depends_on` do `docker-compose.yml` (ou mantenha-o parado). Os backups passam a ser feitos pelos snapshots automáticos do RDS.

14. Segurança: lista de verificação

- Acesso SSH somente por chave e restrito ao seu IP no security group; `fail2ban` ativo.
- Portas 3000 e 5432 fechadas para a internet (ufw + security group); banco Docker publicado apenas em 127.0.0.1.
- `JWT_SECRET` longo e aleatório; senha do banco alterada; arquivo `.env` com permissão 600 (`chmod 600 .env`).
- HTTPS obrigatório (Certbot com redirecionamento) e `APP_URL` em https.
- Atualizações de segurança automáticas do Ubuntu (`unattended-upgrades`) e atualização periódica do Clockfy.
- Backups diários testados (restauração) e cópia fora do servidor (S3) + snapshots EBS.
- Na aplicação: apenas administradores gerenciam faturas, configurações e importação; revise as permissões em **Configurações > Permissões**; revogue chaves de API não usadas em **Perfil > Chaves de API**.

15. Solução de problemas

Sintoma	Causa provável	Como resolver
502 Bad Gateway no navegador	A aplicação não está rodando na porta 3000.	Veja os logs (<code>docker compose logs app / journalctl -u clockfy</code>); confira <code>docker compose ps</code> ou <code>systemctl status clockfy</code> .
Página não abre, nem HTTP	Security group sem 80/443, ufw bloqueando ou DNS não propagado.	<code>sudo ufw status</code> ; regras do security group; <code>dig clockfy.suaempresa.com.br</code> deve retornar SEU_IP.
<code>ECONNREFUSED / password authentication failed</code> nos logs	<code>DATABASE_URL</code> incorreta ou PostgreSQL parado.	Confira usuário/senha/host; <code>docker compose ps db</code> ou <code>systemctl status postgresql</code> .
<code>npm run build</code> é encerrado ("Killed") ou lento	Memória insuficiente na compilação.	Crie o swap (seção 4.2) ou use uma instância maior temporariamente.
<code>permission denied while trying to connect to the Docker daemon</code>	Usuário sem o grupo docker.	<code>sudo usermod -aG docker ubuntu</code> e reconecte a sessão SSH.

Sintoma	Causa provável	Como resolver
Certbot falha na validação	DNS ainda não aponta para o servidor ou porta 80 fechada.	Aguarde a propagação, confira o registro A e o security group, repita o comando.
E-mails não chegam	SMTP não configurado, credenciais erradas ou SES em sandbox.	Procure <code>[mail]</code> nos logs; verifique remetente/domínio no SES; teste com outro provedor.
API responde 401	Chave de API ausente/inválida ou token expirado.	Gere nova chave em Perfil > Chaves de API e envie no header <code>X-Api-Key</code> .
Links de convite/relatório com endereço errado	<code>APP_URL</code> desatualizado.	Ajuste no <code>.env</code> e reinicie a aplicação.
Disco cheio	Imagens Docker antigas ou backups acumulados.	<code>docker image prune -f</code> ; reduza <code>KEEP_DAYS</code> do backup; aumente o volume EBS.

16. Apêndice: arquivos de implantação

deploy/nginx-clockfy.conf

```
# /etc/nginx/sites-available/clockfy
# Proxy reverso do Clockfy. Troque clockfy.exemplo.com.br pelo seu domínio.
# O HTTPS é adicionado automaticamente pelo certbot (certbot --nginx -d clockfy.exemplo.com.br).
server {
    listen 80;
    listen [::]:80;
    server_name clockfy.exemplo.com.br;

    client_max_body_size 25m;           # recibos e imagens (MAX_UPLOAD_BYTES)
    proxy_read_timeout 300s;           # relatórios grandes e importação

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    gzip on;
    gzip_types text/plain text/css application/json application/javascript image/svg+xml;
}
```

deploy/clockfy.service

```
# /etc/systemd/system/clockfy.service (instalação sem Docker)
[Unit]
Description=Clockfy - controle de horas (clone do Clockify)
After=network.target postgresql.service
Wants=postgresql.service

[Service]
Type=simple
User=clockfy
Group=clockfy
WorkingDirectory=/opt/clockfy/server
EnvironmentFile=-/opt/clockfy/server/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node src/index.js
Restart=always
RestartSec=5
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

deploy/backup.sh

```
#!/usr/bin/env bash
# Backup diário do banco do Clockfy (Docker ou PostgreSQL local) com retenção e envio opcional ao S3.
# Uso: sudo /opt/clockfy/deploy/backup.sh (agende no cron: 0 3 * * * /opt/clockfy/deploy/backup.sh)
set -euo pipefail
BACKUP_DIR=${BACKUP_DIR:-/var/backups/clockfy}
KEEP_DAYS=${KEEP_DAYS:-14}
S3_BUCKET=${S3_BUCKET:-} # ex.: s3://minha-empresa-backups/clockfy (opcional)
COMPOSE_DIR=${COMPOSE_DIR:-/opt/clockfy} # onde está o docker-compose.yml
STAMP=$(date +%Y%m%d-%H%M%S)
mkdir -p "$BACKUP_DIR"
FILE="$BACKUP_DIR/clockfy-$STAMP.sql.gz"

if docker compose -f "$COMPOSE_DIR/docker-compose.yml" ps db >/dev/null 2>&1; then
    docker compose -f "$COMPOSE_DIR/docker-compose.yml" exec -T db pg_dump -U clockfy -d clockfy | gzip >
"$FILE"
else
    sudo -u postgres pg_dump -d clockfy | gzip > "$FILE"
fi
echo "backup gerado: $FILE ($(du -h "$FILE" | cut -f1))"
find "$BACKUP_DIR" -name 'clockfy-*.sql.gz' -mtime +"$KEEP_DAYS" -delete
if [ -n "$S3_BUCKET" ]; then aws s3 cp "$FILE" "$S3_BUCKET/" && echo "enviado para $S3_BUCKET"; fi
```

.env.example

```
# Copie para .env (na raiz ou em server/) e ajuste.
PORT=3000
NODE_ENV=development
DATABASE_URL=postgres://postgres:postgres@localhost:5432/clockfy
JWT_SECRET=troque-esta-chave
JWT_EXPIRES_IN=30d
APP_URL=http://localhost:3000

# E-mail (opcional - sem SMTP os e-mails são apenas logados no console)
SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM=Clockfy <no-reply@clockfy.local>

# Limites e jobs
RATE_LIMIT_PER_SECOND=50
SCHEDULER_ENABLED=true
WEBHOOK_TIMEOUT_MS=10000
MAX_UPLOAD_BYTES=10485760
LOG_SQL=false
```